

ArtisanAI

基于 SDXL + ControlNet 的实时风格化图像生成系统

团队：锐意进取 · 参赛者：何瑞清、高毅
项目简介文档 | 版本 v1.0 | 2026 年 8 月 3 日

一、项目背景

1.1 行业背景

以 Stable Diffusion 为代表的扩散模型已成为 AIGC 图像生成的主流技术路线。其中 SDXL (Stable Diffusion XL) 凭借 1024×1024 原生分辨率和显著提升的成图质量，成为开源社区事实上的基座模型。然而，扩散模型的迭代式采样机制决定了它天然「慢」——标准 SDXL 需要 30~50 步去噪，单张图耗时常在十秒量级，难以支撑交互式使用。

与此同时，图像生成的需求正从「文生图」向「图生图」迁移：用户手中已有照片或草图，希望在**保留原有构图与结构**的前提下改变艺术风格，而不是从噪声中随机生成一张无关的新图。这对模型提出了结构可控性的要求。

1.2 待解决的问题

问题	具体表现
推理速度慢	原生 SDXL 需 30~50 步采样，叠加 ControlNet 后单步开销进一步翻倍，交互体验断裂
结构不可控	纯图生图仅靠 strength 调节，强度稍大就丢失原图轮廓，稍小又几乎不改变风格
风格切换成本高	每换一种风格 LoRA 都要重新读盘加载权重，切换耗时数秒，无法快速对比
显存占用大	SDXL + 全尺寸 ControlNet + 多个 LoRA，朴素实现容易超出单卡显存

1.3 项目定位

ArtisanAI 面向 AMD Radeon GPU + ROCm 平台，构建一套**结构可控、风格可切换、响应达到交互级**的图像风格化系统。核心技术选择是三者的组合：

- SDXL-Lightning 对抗蒸馏 LoRA** —— 把采样步数从 50 步压到 8 步，解决速度问题；

- 蒸馏版 Canny ControlNet —— 用边缘图约束生成解决结构保持问题，且采用 160M 参数的蒸馏模型而非 1.25B 的全尺寸版本；
- 多 LoRA 常驻显存 + 动态权重混合 —— 8 种风格全部预加载，切换只改 adapter 权重而不重新加载，解决切换延迟问题。

二、目标用户与应用场景

2.1 目标用户

用户群体	核心诉求	本系统对应能力
自媒体 / 内容创作者	把手头素材图快速转成多种风格配图，用于封面、插图、社交媒体发布	一张输入图，8 种风格秒级切换对比，直接取用
视觉设计师	概念阶段快速探索风格方向，避免在单一方案上过早投入	固定 seed 下切换风格，构图不变、仅风格变化，可控对比
电商 / 营销从业者	商品图、宣传物料的风格化改造，降低外包插画成本	边缘约束保证商品轮廓不失真，风格叠加不改变主体结构
教育 / 展示场景	向非技术受众直观演示扩散模型、ControlNet、LoRA 的作用	Web UI 参数全暴露，并实时输出各阶段耗时拆解
个人普通用户	照片艺术化、头像风格化，零技术门槛	浏览器打开即用，切换风格自动填入推荐提示词

2.2 典型应用场景

场景 A — 单图多风格快速比选

用户上传一张照片，固定 `Seed = 42`、`Strength = 0.7`，依次切换「日式动漫 → 吉卜力 → 油画 → 皮克斯」。由于随机种子与结构约束不变，四张输出的构图完全一致，差异纯粹来自风格，可直接横向比选。风格切换不触发权重重载，等待时间仅为一次推理。

场景 B — 结构保持强度调优

用户通过 `ControlNet Scale`（边缘约束强度）与 `Strength`（重绘强度）两个正交参数，在「忠于原图」与「风格强烈」之间连续调节，找到适合当前素材的平衡点。

场景 C — 性能演示与调优

界面实时输出边缘提取、文本编码、逐步去噪、VAE 解码各环节耗时，可用于在 AMD 平台上定位性能瓶颈、验证优化效果。

2.3 场景约束

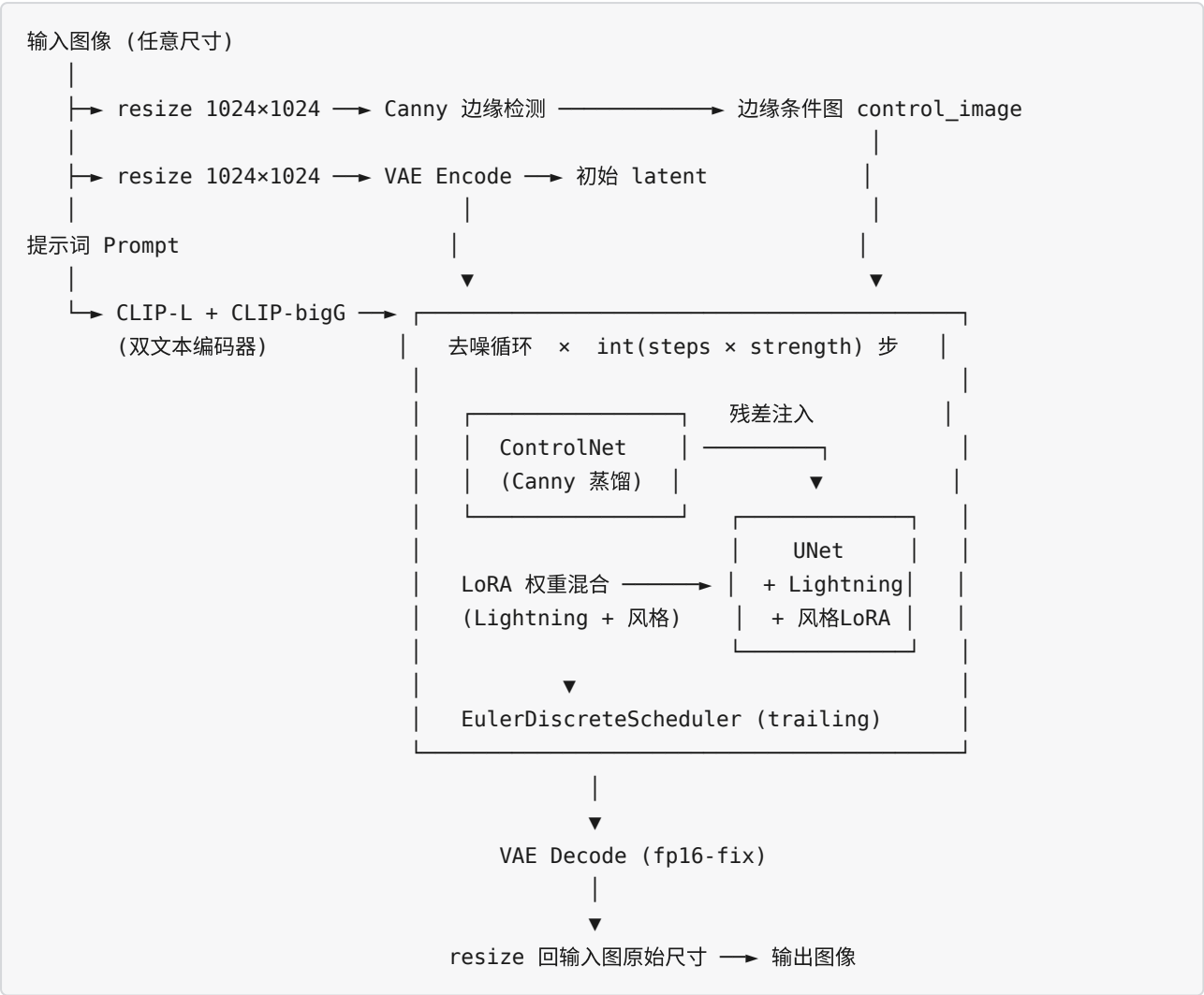
本系统定位于**图生图 (image-to-image)** 风格化，必须提供输入图像，不支持纯文本生成图像；输出分辨率固定按 1024×1024 生成后还原为输入图原始尺寸；单次处理单张图像，暂不支持批量队列。

三、系统架构

3.1 分层架构



3.2 推理数据流



关于实际步数：在图生图模式下，真实执行的去噪步数为 `int(num_inference_steps × strength)`。以界面默认值 `steps=8`、`strength=0.7` 计算，实际仅执行 **5 步**去噪。这是 SDXL-Lightning 蒸馏 LoRA 与图生图机制叠加后的效果，也是本系统达到交互级响应的直接原因。

3.3 关键设计决策

决策	做法	收益
LoRA 全量预加载	启动时 <code>preload_style_loras()</code> 把 8 个风格 adapter 一次性载入显存，切换时只调用 <code>set_adapters()</code> 改权重	切换风格从「读盘 + 加载」的秒级降为近乎零开销；额外显存代价仅 8×9 MB
双 LoRA 权重混合	加速用的 Lightning LoRA 始终以权重 1.0 激活，风格 LoRA 作为第二个 adapter 叠加	加速与风格解耦，任意风格都享有 8 步加速
启动期 Warmup	服务启动时先用内置样例图跑一次完整推理	把 GPU kernel 首次编译开销从用户首次请求转移到启动阶段
全程 fp16 不做 offload	所有组件常驻显存，不启用 <code>enable_model_cpu_offload()</code>	避免每步推理的 Host↔Device 权重搬运，代价是更高的显存占用
边缘图结果缓存	以输入图内容哈希为键缓存最近一张 Canny 边缘图	同图调参、切换风格时跳过整个边缘检测环节——这正是交互式使用的主要模式
分段性能埋点	通过 <code>callback_on_step_end</code> 逐步记录耗时并同步 GPU，同时采集 PyTorch 显存峰值	可精确定位瓶颈环节；生产部署时可关闭以消除同步开销

四、模型与算法介绍

4.1 模型清单

角色	模型	说明
基座模型	<code>stabilityai/stable-diffusion-xl-base-1.0</code> (fp16 variant)	SDXL 1.0，双文本编码器 + 2.6B 参数 UNet，原生 1024×1024
结构控制	<code>diffusers/controlnet-canny-sd-xl-1.0-small</code>	蒸馏版 SDXL ControlNet，160M 参数 / 320 MB fp16，约为全尺寸版本的 1/7.8
边缘检测	OpenCV Canny (<code>controlnet-aux</code> 封装)	传统算子，无需任何模型权重，纯 CPU 计算，不占显存
图像解码器	<code>madebyollin/sd-xl-vae-fp16-fix</code>	修正版 VAE，规避官方 SDXL VAE 在 fp16 下的数值溢出
加速模块	<code>ByteDance/SDXL-Lightning</code> <code>sd-xl_lightning_8step_lora.safetensors</code>	渐进式对抗蒸馏得到的 8 步采样 LoRA
风格模块	<code>ntc-ai/SDXL-LoRA-slider.*</code> × 8	见 4.4 节，单个权重仅 9 MB

4.2 核心算法一：Canny ControlNet 结构约束

ControlNet 通过复制 UNet 编码器分支并接受额外的空间条件图，将条件特征以残差形式注入主干 UNet 的对应层，从而在不破坏基座模型能力的前提下施加空间约束。

条件类型的选择

可用于结构约束的条件类型可按「结构锁定强度」排成一条谱系：Canny（二值硬边缘，最紧）→ Lineart（线稿）→ SoftEdge（连续灰度柔边）→ Scribble（粗略轮廓）→ Depth（仅三维层次，平面内最自由）。锁定越紧，构图还原度越高，但留给风格化的自由度越小。

本项目的选型经历过一次实测驱动的调整。初始版本采用 **SoftEdge**（HED 检测器 + 全尺寸 SoftEdge ControlNet），理论上它位于谱系中段、平衡最佳。但该路线存在一个工程上的硬约束：**SoftEdge/Scribble 类型在开源社区中没有任何蒸馏小模型**——可用的权重均为 1.25B 参数的全尺寸 ControlNet，且所用仓库仅提供 fp32 格式（5 GB），需在加载时现场转换精度。

最终改用 **Canny**，决定性因素是：**Canny 与 Depth 是仅有的两类拥有官方蒸馏版本的条件类型**，而 Canny 与 SoftEdge 同属边缘系，语义最为接近，是唯一可行的降级替换。切换后的收益：

维度	SoftEdge（原方案）	Canny（现方案）	变化
ControlNet 参数量	1.25 B	160 M	↓ 7.8×
权重体积	5004 MB (fp32)	320 MB (fp16)	↓ 15.6×
边缘检测器	HED, 29.4 MB 权重	OpenCV, 无权重	↓ 100%

代价是 Canny 输出二值硬边缘，会连同图像噪声一并保留，重度风格化时可能产生偏生硬的描边感。实践中通过适当调低 `controlnet_conditioning_scale`（0.3~0.4）即可缓解——这正是把该参数暴露到界面上的原因。

约束强度由 `controlnet_conditioning_scale` 控制（默认 0.5）：数值越大越贴合原图边缘，越小则风格化越自由。

关于「参数量减少 7.8 倍」的实际收益：ControlNet 是附加在 UNet 之上的旁路分支，而非替代品。以 SDXL 的 2.6B UNet 为基准，全尺寸 ControlNet 使每步计算量增至约 1.48 倍，换用 160M 的蒸馏版后降至约 1.06 倍——去噪环节的实际提速约为 28%，而非 7.8 倍。真正的计算主体始终是 UNet 本身。本项目对此保持清醒认识：ControlNet 的轻量化是有价值的一环，但采样步数的压缩（见 4.3 节）才是量级上的优化。

4.3 核心算法二：SDXL-Lightning 少步采样

SDXL-Lightning 采用**渐进式对抗蒸馏**（progressive adversarial distillation）：以原始 SDXL 为教师模型，训练学生模型在少数几步内逼近教师多步采样的结果，并引入对抗损失以避免少步采样常见的模糊问题。产出以 LoRA 形式发布，可直接叠加到原始 SDXL 权重上。

使用时必须配套两项设置，缺一不可：

- 调度器：**`EulerDiscreteScheduler` 且 `timestep_spacing="trailing"`。蒸馏训练时采用的是 trailing 时间步排布，推理时不匹配会导致成图明显劣化。
- 引导系数：**蒸馏模型已将引导效果内化到权重中，无需外部引导。本项目取 `guidance_scale = 1.0`，即**完全关闭 classifier-free guidance**。

关闭 CFG 的收益：Diffusers 中当 `guidance_scale > 1` 时会启用 classifier-free guidance，UNet 与 ControlNet 需按 batch=2 各前向一次（正向分支 + 负向分支）。由于 Lightning 已通过蒸馏将引导效果内化，这一路额外计算是纯粹的浪费。将该值设为 `1.0` 后批量减半，去噪环节的计算量直接降低约一半，是本项目单项收益最大的优化。代价是负向提示词失效——对已内化引导的蒸馏模型而言，这一功能本就意义有限。

4.4 核心算法三：多 LoRA 动态混合

基于 PEFT 的多 adapter 机制，系统在启动阶段将加速 LoRA 与全部 8 个风格 LoRA 一次性注册进 pipeline。运行时通过 `set_adapters([加速LoRA, 风格LoRA], adapter_weights=[1.0, scale])` 动态指定生效的组合与权重——权重矩阵按线性组合方式叠加到基座权重的前向计算中，无需任何磁盘 I/O。

#	界面名称	风格	LoRA 仓库
1	日式动漫	Anime	<code>ntc-ai/SDXL-LoRA-slider.anime</code>
2	像素艺术	Pixel Art	<code>ntc-ai/SDXL-LoRA-slider.pixel-art</code>
3	吉卜力工作室	Studio Ghibli	<code>ntc-ai/SDXL-LoRA-slider.Studio-Ghibli-style</code>
4	卡通	Cartoon	<code>ntc-ai/SDXL-LoRA-slider.cartoon</code>
5	油画	Oil Painting	<code>ntc-ai/SDXL-LoRA-slider.oil-painting</code>
6	皮克斯动画	Pixar	<code>ntc-ai/SDXL-LoRA-slider.pixar-style</code>
7	超写实插画	Ultra-realistic Illustration	<code>ntc-ai/SDXL-LoRA-slider.ultra-realistic-illustration</code>
8	千禧独立漫画	2000s Indie Comic	<code>ntc-ai/SDXL-LoRA-slider.2000s-indie-comic-art-style</code>

风格集合按 Hugging Face 下载量降序筛选，并剔除非画风类条目（画质增强、人体姿态、妆容等 slider）。每个风格附带一组推荐提示词，界面切换风格时自动填入。

4.5 关键参数

参数	默认值	作用
<code>strength</code>	0.7	重绘强度，同时决定实际去噪步数 <code>int(steps × strength)</code>
<code>controlnet_conditioning_scale</code>	0.5	边缘约束强度，越大越贴合原图结构
<code>num_inference_steps</code>	8	名义采样步数，与 Lightning 8-step LoRA 匹配
<code>guidance_scale</code>	1.0	CFG 引导系数，取 1.0 即关闭 CFG；蒸馏模型无需外部引导
<code>seed</code>	42	随机种子，固定后可做严格的风格对照实验

五、AMD Radeon GPU / ROCm 适配说明

5.1 运行环境

加速器

项目	配置
GPU	AMD Radeon Graphics × 1 (<code>amd-smi static</code> 的 <code>MARKET_NAME</code> 上报值)
GFX 架构	<code>gfx1100</code> (RDNA 3 / Navi 31, LLVM 目标 <code>amdgcn-amd-amdhsa--gfx1100</code>)
显存容量	49 136 MB (约 48 GB)
PCIe 位置	<code>0000:03:00.0</code> (HIP Device 0)
功耗上限	241 W
VBIOS	<code>00162356</code>

软件栈

项目	配置
ROCm 版本	7.2.1
内核态驱动	<code>amdgpu</code> 6.16.13
管理工具	<code>amd-smi</code> 26.2.2 (+e1a6bc5663)
部署形态	Linux Baremetal (非虚拟化, 容器直通 GPU)
PyTorch 版本	2.9.1 (源码构建, commit <code>ff65f5b</code>)
操作系统	Ubuntu 24.04.4 LTS
Python 版本	容器内 <code>/opt/venv</code> 虚拟环境

主机平台

项目	配置
CPU	AMD EPYC 9334 32-Core Processor × 2 路（Zen 4，Family 25 Model 17）
核心 / 线程	64 物理核 / 128 逻辑线程
NUMA 拓扑	2 节点（node0: CPU 0-31,64-95；node1: CPU 32-63,96-127）
缓存	L2 64 MiB / L3 256 MiB（8 组）
系统内存	503 GiB

本项目运行在**全 AMD 平台**上——AMD EPYC 9334 作为主机 CPU，AMD Radeon GPU 作为推理加速器，通过 ROCm 7.2.1 软件栈打通。CPU 侧 128 线程与 503 GiB 内存为模型权重加载、图像编解码等主机侧任务提供了充裕资源，不构成瓶颈。

5.2 适配策略：零代码改动迁移

ROCm 版 PyTorch 通过 HIP 运行时对 CUDA 编程模型做了语义级映射，`torch.cuda.*` 系列 API 在 ROCm 构建下直接指向 AMD GPU（本环境中即 HIP Device 0）。因此本项目的全部设备相关代码——包括 `.to("cuda")`、`torch.cuda.synchronize()`、`torch.Generator(device="cuda")`——**无需任何修改即可在 ROCm 7.2.1 平台上运行**，源码保持与 NVIDIA 平台完全一致，不存在任何平台分支代码或条件编译。

这一策略的价值在于：项目可在单一代码库上同时支持两个硬件平台，降低维护成本，也便于做跨平台性能对比。

5.3 精度与数值稳定性适配

本项目采用 fp16 全流程，这一选择与目标硬件的架构特性直接匹配：**gfx1100（RDNA 3）引入了 WMMA（Wave Matrix Multiply-Accumulate）矩阵指令，原生支持 fp16 数据类型的矩阵乘累加运算**。扩散模型的计算量绝大部分集中在 UNet 与 ControlNet 的卷积和注意力矩阵乘上，以 fp16 运行可直接命中这条硬件加速通路。

- **全流程 fp16**。所有模型组件（UNet、ControlNet、双文本编码器、VAE）统一以 `torch_dtype=torch.float16` 加载，充分利用 RDNA 3 的半精度矩阵算力并将显存占用减半。
- **VAE 溢出规避**。SDXL 官方 VAE 在 fp16 下存在已知的数值溢出问题，会产生黑图或色块。本项目改用 `madebyollin/sd-xl-vae-fp16-fix`，该权重经过重训以适配 fp16 动态范围，是保证 fp16 全流程可用的必要条件。
- **基座模型 fp16 变体**。加载时指定 `variant="fp16"`，直接下载半精度权重，避免运行时转换的额外开销与显存峰值。

5.4 显存管理

系统采用「全部常驻、不做 offload」策略：

- SDXL 主干、蒸馏版 ControlNet、VAE、Lightning LoRA 与 8 个风格 LoRA 全部驻留显存（Canny 检测为纯 CPU 算子，不占显存）；
- 风格 LoRA 单个仅 9 MB，8 个合计约 72 MB，相对基座模型可忽略，这使得「全部预加载」的设计在显存上完全可行；
- 不启用 `enable_model_cpu_offload()`，避免每步推理都发生 Host↔Device 权重搬运——在 PCIe 带宽受限的场景下，offload 带来的延迟往往超过计算本身。

本平台 48 GB 的显存容量为该策略提供了充分空间：即使在 fp16 全精度、ControlNet 与全部 LoRA 常驻的配置下，显存仍有大量余量，无需为容量做任何妥协（如量化、分块解码或分层 offload）。这也是 5.5 节中量化方案被判定为「无必要」的直接前提。

显存实测占用：**PyTorch 张量峰值 9.91 GB，分配器保留 11.21 GB**（详见 5.6 节）。相对 48 GB 的可用容量，占用率约 23%，余量充足。

5.5 量化方案的取舍

项目实现中保留了基于 bitsandbytes NF4 的 4-bit 量化通路（`gen_image.py` 中的 `USE_NF4` 开关），可将 UNet 权重压缩至 4-bit 以降低显存占用。但经实测评估后**默认关闭**，原因有二：

- **收益方向不匹配**。量化的核心价值是降低显存占用，而本平台 48 GB 显存对本项目而言远超需求（实测占用仅约 11 GB），显存从来不是约束条件。
- **存在负收益**。4-bit 权重在每次前向时都需反量化回计算精度，这部分开销在本项目的负载特征下无法被显存节省所补偿，端到端延迟反而上升。

该通路作为低显存设备（如 8~12 GB 消费级显卡）的备选方案保留在代码中，可通过单个布尔开关启用，不影响主路径。

5.6 性能表现

测试条件：输入图 1024×1024，`steps=8`、`strength=0.7`（实际执行 5 步去噪）、`guidance_scale=1.0`、`controlnet_conditioning_scale=0.5`、`seed=42`。数据取自服务预热完成后的稳态运行，由系统内置的分段计时与显存统计直接输出。

指标	实测值	占比	说明
LoRA 切换（ <code>set_adapters</code> ）	48.1 ms	2.3%	纯显存内权重系数调整，无磁盘 I/O
Canny 边缘提取	0.01 s	0.5%	OpenCV 算子，CPU 侧计算，无模型权重
Setup（文本编码 + 首步）	0.60 s	29.3%	双 CLIP 编码 + VAE Encode + 第 1 步去噪
单步去噪耗时	0.246 s/step	48.0%	UNet + ControlNet 单步前向，CFG 已关闭（batch=1），第 2~5 步共 4 步
VAE 解码	0.40 s	19.5%	fp16-fix VAE，latent → 1024×1024 图像
Pipeline 总耗时	1.98 s	96.6%	编码 + 5 步去噪 + 解码
端到端单图耗时	2.05 s	100%	从请求到图像返回的完整链路
显存峰值（PyTorch allocated）	9.91 GB	—	模型权重 + 中间激活的实际张量占用
显存峰值（PyTorch reserved）	11.21 GB	—	分配器缓存池； <code>amd-smi</code> 的进程级读数在此基础上再加 HIP 运行时上下文开销

结论：1024×1024 图生图端到端 2.05 秒，达到交互级响应。开销分布为——**去噪 77%**（Setup 内含首步）、**VAE 解码 20%**、LoRA 切换与边缘提取合计不足 3%。

两点值得注意：其一，**LoRA 切换仅 48 毫秒**，验证了「全量预加载 + 权重混合」设计的有效性——若改为传统的读盘重载方式，此项将是秒级开销，8 种风格的快速比选将无法成立。其二，**在步数已压缩至 5 步后，固定开销（Setup 中的编码部分 + VAE 解码，合计约 0.75 s）已接近去噪本身的量级**，这意味着继续削减采样步数的边际收益正在快速递减，后续优化重心应转向编解码环节（见 5.8 节）。

5.7 已完成的平台侧优化

#	优化项	说明
1	8 步蒸馏采样	Lightning LoRA + trailing 调度器，采样步数由 50 步降至 8 步（图生图下实际 5 步）
2	LoRA 常驻显存	风格切换零磁盘 I/O，仅调整 adapter 权重
3	fp16 全流程	配合 fp16-fix VAE 保证数值稳定
4	启动期 Warmup	GPU kernel 首次编译开销前移至服务启动阶段，用户首次请求即达稳态性能
5	免 offload 常驻	消除逐步的 Host↔Device 权重搬运
6	关闭 CFG	<code>guidance_scale = 1.0</code> ，UNet 与 ControlNet 批量由 2 降为 1，去噪计算量减半（详见 4.3 节）
7	边缘图结果缓存	缓存最近一张输入图的 Canny 边缘图，同图调参或切换风格时跳过边缘检测。该机制在早期采用 HED 神经网络检测器时收益显著；改用 OpenCV Canny 后单次提取已降至 0.01 s，此项现为边际优化
8	MIOpen 卷积算法自动调优	启用 <code>torch.backends.cudnn.benchmark</code> ，在 ROCm 上映射为 MIOpen 的 exhaustive find 模式，为固定输入形状实测选出最优卷积算法。本项目分辨率恒为 1024×1024、关闭 CFG 后 batch 恒为 1，形状完全稳定，满足该模式的生效前提；一次性调优开销由启动 warmup 吸收，且 MIOpen 会将结果持久化到本地数据库，服务重启后无需重新调优

5.8 后续优化方向

- 1. **轻量化 VAE 解码**——实测显示 VAE 解码占端到端耗时约 20%，在总步数已压缩到 5 步的前提下，它已成为仅次于去噪的第二大开销。可引入 TAESD 将该环节从数百毫秒降至数十毫秒，或采用两段式策略：预览阶段用 TAESD 快速出图，用户确认后再以完整 VAE 输出终图。
- 2. **文本嵌入缓存**——同一提示词重复生成时跳过双 CLIP 编码。该开销包含在 Setup 环节内，实测约占端到端耗时的 17%。
- 3. **关闭计时同步**——生产模式下移除逐步的 `torch.cuda.synchronize()`。该同步用于精确采集每步耗时，但会打断 CPU/GPU 流水，性能数据采集完成后可关闭。
- 4. **更激进的蒸馏档位**——改用 4-step Lightning LoRA，实际去噪步数由 5 步进一步降至 2~3 步，需评估画质损失是否可接受。
- 5. **torch.compile**——对 UNet 与 ControlNet 做图编译，通常可获得 20~30% 收益。但需先验证两点：与 `set_adapters` 动态切换是否触发重编译；以及 Triton 后端在 gfx1100（RDNA 3）上的成熟度——该后端在 CDNA 架构上的优化更为充分。

需要说明的是，**本项目不考虑任何以降低显存为目标的优化手段**（量化、CPU offload、VAE 分块解码等）。在 48 GB 显存、实测占用约 11 GB 的前提下，这类手段没有收益空间，且普遍以牺牲速度为代价。优化方向集中于降低计算量与提升执行效率。